

Sprawozdanie obliczenia naukowe

Lista 5

Jakub Kowal

Zapis macierzy

Reprezentacja macierzy

Macierze są przechowywane w strukturze **BlockMatrix** jako wektor bloków diagonalnych **As** oraz wektory bloków nad i poddiagonalnych **Cs** i **Bs**. Każdy blok ma rozmiar $l \times l$, a liczba bloków na diagonalu to $v = \frac{n}{l}$. Dzięki temu możemy zmniejszyć użycie pamięci z $O(n^2)$ do $O(3 \times v \times l^2)$, gdzie $v \times l$ daje nam n , więc otrzymujemy $O(n \times l)$. Biorąc pod uwagę, że l jest stałe możemy przyjąć $O(n)$ jako złożoność pamięciową.

Operacje odczytu i zapisu używają pomocniczych funkcji **matrix_get** i **matrix_put**, które zwracają lub ustawiają wartości w konkretnych blokach znając współrzędne w oryginalnej macierzy. Mają złożoność obliczeniową $O(1)$.

Eliminacja Gaussa

Implementacja

Implementacja funkcji **Gauss** przeprowadza eliminację tylko w obrębie zapisywanych bloków. Taka implementacja jest możliwa dzięki specyficznej naturze naszej macierzy, która oznacza, że nie mamy potrzeby ingerowania w puste pola poza blokami **A**, **B** i **C**. Dla każdego wiersza obliczany jest mnożnik I (Nie ma potrzeby zapisywać w pamięci tych mnożników, więc korzystamy z pojedynczej zmiennej) i wykonywana jest aktualizacja elementów w blokach oraz wektora prawej strony b .

Dzięki specyficznej postaci macierzy algorytm ten może wykonywać się tylko na wartościach przechowywanych w blokach, ponieważ reszta i tak byłaby zerowana przez zera spoza bloków. Pozwala nam to niesamowicie obniżyć złożoność z $O(n^3)$ do $O(n)$, ale to zostanie wytłumaczone w późniejszych punktach.

Eliminacja Gaussa z pivotingiem

Funkcja **Gauss_pivot** działa tak jak funkcja **Gauss**, ale dodatkowo wyszukuje największy element w kolumnie w zakresie pivotowania (ograniczonym przez

nasze bloki) i wykonuje zamianę wierszy za pomocą **matrix_swap**. Wykonuje też zamianę danych w wektorze b.

Rozkład LU

Implementacja

Funkcja **LU** zamienia macierz w postać, w której dolna część (poniżej diagonalnej) zawiera współczynniki L (mnożniki w postaci zmiennej I), a górna część macierz U. W implementacji mnożniki zapisywane są w macierzy w miejscu pól, które były zerowane przez eliminację Gaussa. Jest to praktycznie jedyna różnica względem eliminacji Gaussa.

Rozkład LU z pivotingiem

Funkcja **LU_pivot** wykonuje wyszukiwanie pivotu analogicznie do pivotingu w eliminacji Gaussa i zamianę wierszy (funkcja **matrix_LU_swap**, która operuje na szerszym zakresie indeksów niż zwykły **matrix_swap**). Następnie zapisuje mnożniki i aktualizuje elementy macierzy oraz wektore b.

Obliczanie

Implementacja

Funkcja **solve_gauss** wykonuje obliczanie x od x_n do x_1 na macierzy w postaci po eliminacji Gaussa (przyjmuje macierz i wektor prawej strony). Na wykładzie zostało opisane to wzorem:

$$x_k = \frac{b_k - \sum_{j=k+1}^n a_{kj} x_j}{a_{kk}}$$

Z kolei **solve_LU** najpierw modyfikuje wektor b, żeby wziąć pod uwagę mnożniki, które w eliminacji Gaussa na bieżąco zmieniały b, a w rozkładzie LU dopiero teraz trzeba wziąć pod uwagę. Następnie wykonuje po prostu **solve_gauss** dla części U.

Złożoność

Złożoność czasowa

```
function Gauss(matrix::BlockMatrix,b::Vector)
    for k in 1:matrix.n-1
        for i in k+1:min(matrix.n, k + 2*matrix.l)
            I=matrix_get(matrix,i,k)/matrix_get(matrix,k,k)
```

```

        matrix_put(matrix,i,k,0.0)
    for j in k+1:min(matrix.n, k + 2*matrix.l)
        val=matrix_get(matrix,i,j)- I * matrix_get(matrix,k,j)
        matrix=matrix_put(matrix,i,j,val)
    end
    b[i]=b[i]-I*b[k]
end
end
return matrix, b
end

```

Jak widać w kodzie tej funkcji, korzysta ona z trzech zagnieżdżonych jedna w drugiej pętli. Pierwsza z nich wykonuje się n razy, a kolejne $2l$ razy. Daje nam to złożoność $O(n \times 2l \times 2l)$, ale biorąc pod uwagę stałe l otrzymujemy: $O(n)$. Warto dodać również, że wszystkie funkcje wywoływane wewnątrz pętli są rzędu $O(1)$. Wersja Gaussa z częściowym wybieraniem elementu głównego korzysta z dodatkowej pętli rzędu $O(l)$, także biorąc pod uwagę stałe l możemy stwierdzić, że również jest rzędu $O(n)$.

W przeciwieństwie do pełnej eliminacji Gaussa na macierzach gęstych, która ma złożoność $O(n^3)$, wykorzystanie struktury blokowej możliwe, dzięki specyficznej budowie naszej macierzy znacząco pozwala nam zmniejszyć złożoność obliczeniową.

```

function LU(matrix::BlockMatrix)
    for k in 1:matrix.n-1
        for i in k+1:min(matrix.n, k + 2*matrix.l)
            I=matrix_get(matrix,i,k)/matrix_get(matrix,k,k)
            matrix_put(matrix,i,k,I)
            for j in k+1:min(matrix.n, k + 2*matrix.l)
                val=matrix_get(matrix,i,j)- I * matrix_get(matrix,k,j)
                matrix=matrix_put(matrix,i,j,val)
            end
        end
    end
    return matrix
end

```

Funkcja obliczająca rozkład LU, działa na takiej samej zasadzie jak Gauss z drobnymi poprawkami. Oznacza to taką samą złożoność $O(n)$ zarówno dla wersji z częściowym wyborem elementu głównego, jak i bez wyboru.

```

function solve_gauss(matrix::BlockMatrix,b::Vector{Float64})
    x = zeros(Float64, matrix.n)
    for i in matrix.n:-1:1
        sum_ax = 0.0
        for j in i+1 : min(matrix.n, i + 2*matrix.l)
            sum_ax += matrix_get(matrix, i, j) * x[j]
        end
        x[i] = (b[i] - sum_ax) / matrix_get(matrix, i, i)
    end

    return x
end

```

Funkcja obliczająca $Ax=b$ dla macierzy po eliminacji Gaussa korzysta z dwóch pętli. Jedna wykonuje się n razy, a jej wewnętrzna pętla $2 \times l$. Oznacza to złożoność rzędu $O(n \times 2l)$, czyli $O(n)$.

```

function solve_LU(matrix::BlockMatrix, b::Vector{Float64})
    for k in 1:matrix.n-1
        for i in k+1:min(matrix.n, k + 2*matrix.l)
            b[i] = b[i] - matrix_get(matrix, i, k) * b[k]
        end
    end
    return solve_gauss(matrix, b)
end

```

Funkcja obliczająca macierz po rozkładzie LU działa na podobnej zasadzie co funkcja obliczająca eliminację Gaussa, jedynie z dodatkiem mnożenia wektora b poprzez wcześniej wyliczone mnożniki L . Jej złożoność to $O(n \times 2l + n)$, czyli $O(n)$.

Złożoność pamięciowa

Macierz jest przechowywana jako $v = n/l$ bloków rozmiaru $l \times l$, stąd pamięć zajęta przez bloki diagonalne to $v \cdot l^2 = n l$. Podobnie bloki nad- i poddiagonalne też razem dają rząd $O(nl)$. Wektory prawej strony i rozwiązania są długości n . Zatem całkowite wymagania pamięciowe to

$$M(n) = O(nl).$$

Dla stałego i małego l jest to liniowe w n , znacząco mniej niż $O(n^2)$ dla pełnej macierzy gęstej.

Wyniki i wnioski

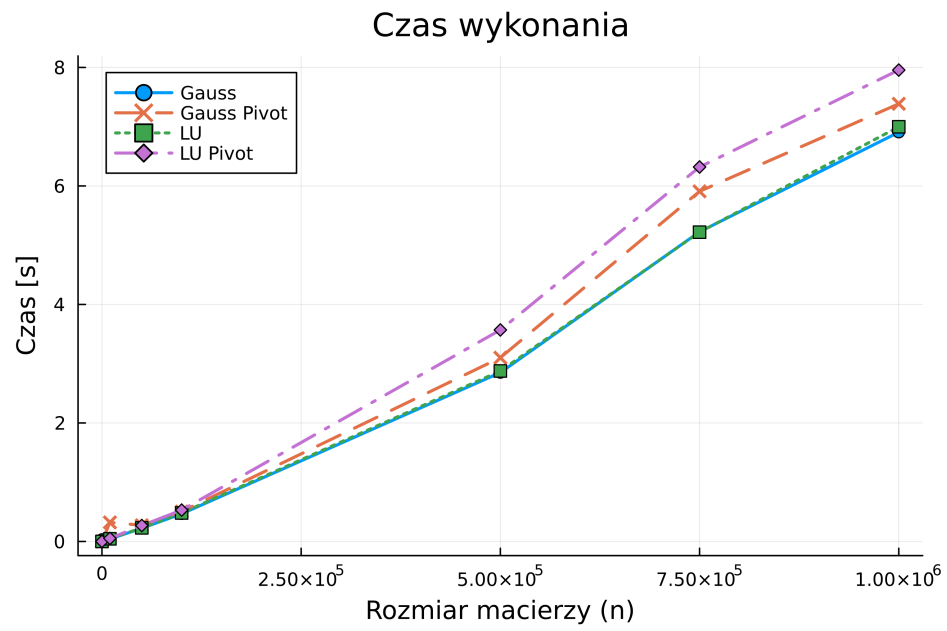
W pliku z eksperymentami (**Usage.jl**) wykonane zostały pomiary czasu i alokacji pamięci dla czterech wariantów: **Gauss**, **Gauss Pivot**, **LU** i **LU Pivot**. Wykresy wyników zostały dołączone jako **time_graph.png** i **mem_graph.png**.

Obserwacje istotne dla wniosków: - Implementacje operują wewnątrz bloku/pasma, stąd złożoność rośnie liniowo z n mnożoną przez l^2 , a nie jak n^3 . - **LU** zapisuje mnożniki **L** w macierzy (dolna trójkątna część jest wypełniana wartościami różnymi od zera), co zmienia strukturę danych po obliczeniach w porównaniu do **Gauss**, który ustawia część eliminowaną na zera. W praktyce jednak pamięć bloków jest wcześniej zaalokowana, więc różnica pamięci bezpośrednio po operacji zależy od dodatkowych alokacji pomocniczych. - Wpływ pivotingu: pivoting wymusza dodatkowe zamiany wierszy (funkcje **matrix_swap** i **matrix_LU_swap**), co zwiększa liczbę odczytów/zapisów i nieznacznie narzut czasowy; **matrix_LU_swap** operuje na nieco szerszym zakresie indeksów, co zwiększa koszty przy pivotingu w **LU**.

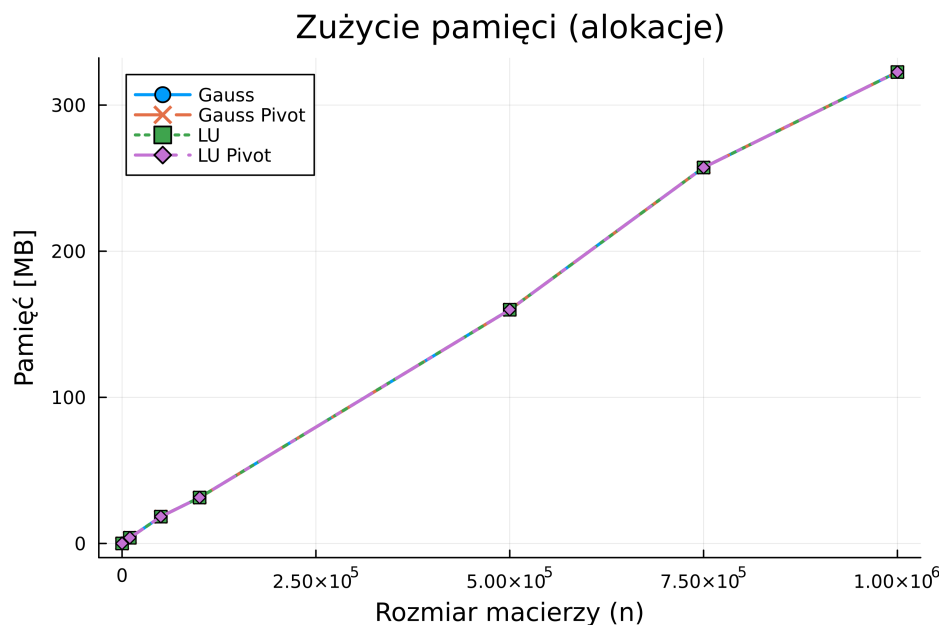
Proponowane optymalizacje i uwagi metodyczne: - Jeżeli chcemy minimalizować alokacje, należy unikać kopiowania wektora w funkcjach rozwiązujących — w aktualnej wersji **solve_LU** modyfikacja wektora jest wykonywana in-place (komentarz z **y = copy(b)** został zakomentowany), co redukuje jedną dużą alokację. - Dla stabilniejszych i powtarzalnych pomiarów rekomenduję użycie pakietu **BenchmarkTools.@btime** zamiast **@timed** — daje lepsze statystyki alokacji i czasu. - Można też ograniczyć zwroty z **matrix_put** (nie musi zwracać macierzy), aby zmniejszyć narzut wywołań funkcyjnych.

Wyniki

Wyniki czasowe



Wyniki pamięciowe



Wnioski

Algorytmy korzystają z reprezentacji blokowej, co pozwala na istotne przyspieszenie i ograniczenie pamięci w porównaniu do pełnej macierzy. Dzięki stałemu l koszty są asymptotycznie rzędu $O(n)$ w czasie i $O(n)$ w pamięci, co czyni podejście niesamowicie efektywnym dla dużych n .

Ciekawostka

Tą animację wygenerował ChatGPT wykorzystując pakiet `animate` oraz `tikz`. Do generowania kolejnych klatek użył osobnego skryptu pythonowego.

